

Code Assessment of the Strategy Deployment Zap Smart Contracts

PUBLIC

Date [June 05, 2026](#)

Produced for



By



Contents

1	Executive Summary	3
2	Assessment Overview	4
3	System Overview	5
4	Trust Model	6
5	Terminology	7
6	Findings	8
7	Limitations and use of report	10

1 Executive Summary

Dear Makina team,

Thank you for trusting us to help Makina with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of Strategy Deployment Zap according to [Scope](#) to support you in forming an opinion on their security risks.

Makina implements a zipper contract that fully deploys a new strategy, both core and periphery. The contract relies on the existing factories and operates under a timelock.

The most critical subjects covered in our audit are functional correctness, access control, and the integrity of the timelock-protected scheduling, execution, and cancellation flows. Security regarding all the aforementioned subjects is high. We did not identify any issues affecting these subjects.

The general subjects covered are code clarity and gas efficiency. Security regarding both is high.

In summary, we find that the codebase provides a high level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,
ChainSecurity

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the Strategy Deployment Zap repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

Version	Date	Commit Hash	Note
1	6 May 2026	7085ed0428f6f28e9bb3a5a1ba980c3a76600262	Initial review
2	4 June 2026	ecd97a9db2f2729ab8af7ce16f3c00c91519ecc0	Final Version

For the Solidity smart contracts, the compiler version `0.8.28` was chosen.

```
src/  
├─ HubStrategyDeploymentZap.sol  
└─ StrategyDeploymentZapBase.sol
```

2.1.1 Excluded from scope

Any file not explicitly listed in the Scope section is out of scope. In particular, external libraries (e.g. OpenZeppelin), the Makina core and periphery factories invoked by the zap, the AccessManager controlling factory permissions, and tests are excluded.

Additionally, all configurations and operational usage are out of scope of this review.

3 System Overview

This system overview describes the version (**Version 2**) of the contracts as defined in the [Assessment Overview](#).

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

Makina provides `HubStrategyDeploymentZap`, a contract that performs the creation of a Makina strategy on the Hub chain in a single timelock-gated transaction. The zap calls into the Makina core and periphery factories to deploy the Machine, Caliber, CaliberMailbox(es), bridge adapters, and periphery modules, and to wire AccessManager roles on the resulting contracts.

The zap does not deploy code itself or hold any AccessManager admin role. It must be granted `STRATEGY_DEPLOYMENT_ROLE` on the AccessManager so that the core and periphery factories accept its calls.

3.1 StrategyDeploymentZapBase

The abstract base implements a per-payload schedule keyed by `(executor, payloadHash)`. The owner schedules a deployment with `scheduleDeployment(executor, payloadHash, delay)` and may cancel an unexecuted schedule with `cancelDeployment`. An entry point consumes a schedule via `_consumeSchedule(msg.sender, keccak256(msg.data))`, which checks that the schedule exists, the delay has elapsed, and it has not been executed. On success, the schedule's timepoint is overwritten with `EXECUTED_TIMEPOINT = type(uint256).max`, permanently marking it consumed.

3.2 HubStrategyDeploymentZap

Two entry points orchestrate the deployment, each consuming its own schedule:

- `createMachine`: fresh deployment of a Machine.
- `createMachineFromPreDeposit`: deployment that migrates an existing `PreDepositVault` into the new Machine.

Both take the Machine, Caliber, governance, and bridge adapter init parameters, plus a periphery sub-struct that lets the zap either deploy fresh depositor/redeemer/fee-manager modules from beacons or accept pre-deployed addresses (mixing both for the same module reverts). Each entry point then calls `IHubPeripheryFactory` to create any requested periphery modules, calls `IHubCoreFactory.createMachine(...)` (or `createMachineFromPreDeposit(...)`), and finally calls `IHubPeripheryFactory.setMachine(module, machine)` for each freshly-deployed periphery module to bind it to the Machine.

4 Trust Model

4.1 Roles and Privileges

4.1.1 Owner

The zap's owner role, set via OpenZeppelin's `Ownable` at construction.

Trust Level: Fully trusted.

Capabilities: Schedules and cancels deployments via `scheduleDeployment` and `cancelDeployment`. Can transfer or renounce ownership. The owner controls which payloads are eligible for execution and which executor is allowed to trigger them. The delay parameter is owner-chosen and may be zero.

A malicious owner could schedule a deployment with adversarial Machine, Caliber, or bridge adapter init parameters. The delay-based scheduling is intended to provide on-chain visibility, not to constrain the owner's authority. The owner cannot, however, cause a deployment to execute with parameters different from what was scheduled.

4.1.2 Scheduled Executor

The address pinned in a given schedule. Must match `msg.sender` of the entry point.

Trust Level: Partially trusted.

Capabilities: Can call `createMachine` or `createMachineFromPreDeposit` with the exact scheduled payload after the configured delay has elapsed. Cannot deviate from the payload and cannot execute unscheduled deployments. The executor's only meaningful power is choosing whether and when to execute the deployment within the schedule's window.

4.2 AccessManager Dependency

The zap contract itself must be granted `STRATEGY_DEPLOYMENT_ROLE` on the AccessManager configured in the Makina core.

4.3 Upgradeability

The zap is not upgradeable. The factory references are immutable.

5 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- **Likelihood** represents the likelihood of a finding to be triggered or exploited in practice
- **Impact** specifies the technical and business-related consequences of a finding
- **Severity** is derived based on the likelihood and the impact

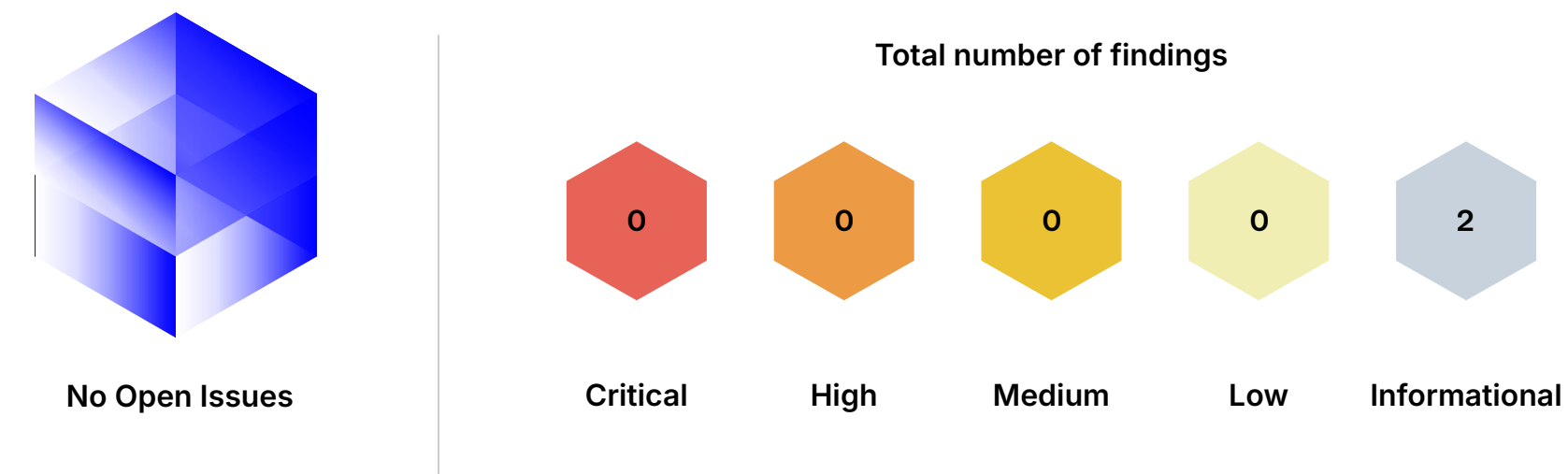
We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

6 Findings

In this section, we describe the findings identified during the course of the engagement. The findings are categorized by severity as explained in the Terminology section. Below we provide an overview of the total number of findings per severity, as well as the number of open issues.



6.1 Overview

The following table lists all identified findings along with their severity and current status. A finding is considered resolved once it has been fully corrected or the specification has been changed accordingly. Findings with any other status, including partial fixes, acknowledgements, and accepted risks, remain open.

Info	#001	cancelDeployment Takes the Full Payload to Recompute Its Hash	Code Corrected	✓
Info	#002	scheduleDeployment Computes block.timestamp + delay Three Times	Code Corrected	✓

6.2 Descriptions

#001 `cancelDeployment` Takes the Full Payload to Recompute Its Hash

Informational

Version 1

Code Corrected ✓

`cancelDeployment` takes `bytes calldata payload` and hashes it. The `scheduledDeployments` mapping is keyed by that hash, which is already emitted as the indexed `payloadHash` topic of `DeploymentScheduled`. Passing the hash directly would be more efficient.

The same observation applies in principle to `scheduleDeployment`, though there the payload bytes also let external parties (e.g., the deployer) decode the scheduled parameters, making the owner's commitment transparent on-chain.

Code Corrected

`cancelDeployment` now takes `bytes32 payloadHash` directly instead of recomputing it from the full payload.

#002 `scheduleDeployment` Computes `block.timestamp + delay` Three Times

Informational

Version 1

Code Corrected ✓

`scheduleDeployment` performs the same addition in three places. The local variable `timepoint` is assigned the result of `block.timestamp + delay` and used only for the `EXECUTED_TIMEPOINT` comparison. The storage write and the event emission re-evaluate `block.timestamp + delay` instead of reusing `timepoint`.

Code Corrected

`timepoint` is now reused for the storage write and the event emission, eliminating the redundant additions.

7 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.